



# Stellar Bootcamp

Stellar & Soroban Bootcamp

Prof. Josiel Maimone de Figueiredo

Maio/2026

# Our Bootcamp Classes

**Class 1** (05/05) — Stellar Fundamentals

**Class 2** (12/05) — Custom tokens and Claimable Balances

**Class 3** (19/05) — Smart Contracts with Soroban

**Class 4** (26/05) — Integration and Final dApp

**Class 1** (05/05)

Stellar Fundamentals

**REVIEW**

# Class 1 – Stellar Fundamentals

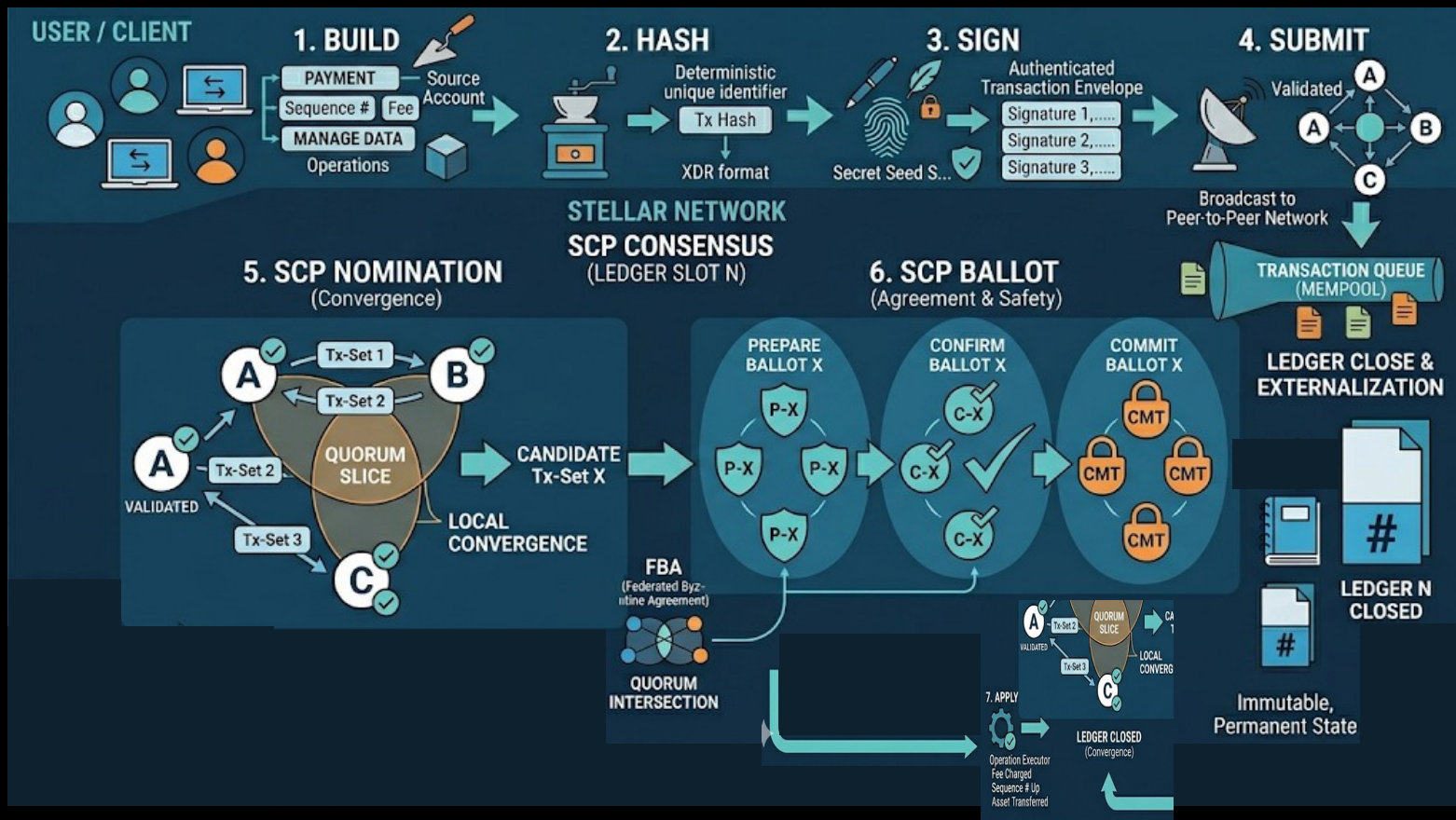
## Your “first” step on Stellar

- Purpose and vision of the Stellar blockchain.
- Differences between Stellar and other blockchains.
- Accounts, public keys, and private keys.
- Testnet and Mainnet.
- Environment setup with Stellar.

## Transactions and security

- Transaction structure on Stellar.
- Cryptographic signatures and account control.
- Simulation of Soroban transactions.
- Complete flow: construction, signing, simulation, and submission

# Stellar Fundamentals



## Class 2 (12/05)

Custom tokens and Claimable Balances

REVIEW

# Class 2 – Custom tokens and Claimable Balances

## Baguncinha journey

- Baguncinha transactions
- Stellar RPC journey

## Baguncinha token

- Smart token
- Claimable clawback

# Class 3 – Smart Contracts with Soroban

Before coffee:

## Rust ultrafast journey

- Baguncinha transactions
- 

After coffee:

## Smart contracts in Soroban

- Smart token
- Claimable clawback



# Soroban and Smart Contracts



Jão's Baguncinha epic continues

“The Stellar smart contracts meets BAGU”

# Stellar & Rust



## Jão's Baguncinha

### Prologue:

- **X-Bagunça** is a kind of X-Tudo sandwich that exists on Cuiabá city.
- **Baguncinha** is a small X-Bagunça.
- Jão has as 15 years baguncinha's snack bar.



# Stellar & Rust



João is learning Rust:

- 1) João tracks Baguncinha inventory with his first Rust variables: no Soroban without Rust first.
  - The lesson starts with **variables**, which are **immutable by default** in Rust — you must write **mut** to allow changes.
  - This forces João to think about what data changes and what does not.

Rust code:

```
// João's first Rust program: Baguncinha inventory
fn main() {
    // Immutable - cannot change after binding
    let stand_name = "João's Stand";

    // Mutable - stock changes during the day
    let mut baguncinhas_left: u32 = 100;
    baguncinhas_left -= 3; // sold 3
    baguncinhas_left += 50; // restocked

    // Compile-time constants - always UPPER_SNAKE_CASE
    const PRICE_PER_DOG: f64 = 2.50;
    const MAX_CAPACITY: u32 = 500;

    // Shadowing - reuse name, optionally change type
    let total = baguncinhas_left;
    let total = total as f64 * PRICE_PER_DOG; // now f64

    println!("Stand: {stand_name}");
    println!("Stock: {} baguncinhas", baguncinhas_left);
    println!("Value: ${:.2}", total);
    println!("Max: {} baguncinhas", MAX_CAPACITY);
}
```

# Stellar & Rust



João is learning Rust:

## 2) João groups his menu data with tuples and arrays

- Tuples hold values of different types.
- Arrays hold values of the same type with a fixed length known at compile time.
- Slices borrow a portion of an array without copying.

Rust code:

```
fn main() {  
    // Tuple: (item_name, quantity, price)  
    let order: (&str, u32, f64) = ("Classic Bagunca", 2, 2.50);  
  
    // Access by index  
    println!("Item: {}, Qty: {}, Price: ${}", order.0, order.1, order.2);  
  
    // Destructure into named variables  
    let (_item, qty, price) = order;  
    let total = qty as f64 * price;  
    println!("Total: ${:.2}", total);  
  
    // Array: fixed menu, size known at compile time  
    let menu: [&str; 4] = [  
        "Classic", "Chili Bagunca", "Cheese Bagunca", "Veggie Bagunca"  
    ];  
    println!("Menu items: {}", menu.len());  
  
    // Slice: borrow part of the array (no copy)  
    let baguncinha_options: [&str] = &menu[0..2];  
    for item in baguncinha_options {  
        println!("  Menu option: {item}");  
    }  
  
    // Array of prices matching menu order  
    let prices: [f64; 4] = [2.50, 3.00, 3.25, 2.75];  
    let most_exp = prices.iter().cloned()  
        .fold(f64::NEG_INFINITY, f64::max);  
    println!("Priciest: ${most_exp}");  
}
```

# Stellar & Rust



Jão is learning Rust:

3) Jão's secret recipe can only be in one place — ownership explained.

- Ownership is what makes Rust memory-safe with no garbage collector. Every value has exactly one owner.
- When ownership moves, the original is gone.
- When the owner goes out of scope, the value is dropped (memory freed).
- This eliminates use-after-free, double-free, and dangling pointers.

Rust code:

```
fn main() {  
    // String lives on the heap — has one owner  
    let recipe = String::from("Jão's secret sauce");  
  
    // MOVE: ownership transferred to nephew  
    let nephew_got = recipe;  
    // println!("{recipe}"); // ERROR: value moved!  
  
    // CLONE: explicit deep copy — both owners exist  
    let recipe2 = String::from("chili formula");  
    let backup = recipe2.clone(); // new allocation  
    println!("Original: {recipe2}, Backup: {backup}");  
  
    // COPY: primitive types are copied implicitly  
    let stock: u32 = 100;  
    let display = stock; // copy — stock still valid  
    println!("Stock: {stock}, Display: {display}");  
  
    // DROP: memory freed when scope ends  
    {  
        let temp = String::from("temp order");  
        println!("Temp: {temp}");  
    } // temp dropped here — no memory leak  
  
    // Move into function — nephew_got no longer usable  
    process(nephew_got);  
}  
  
fn process(order: String) {  
    println!("Processing: {order}");  
} // order dropped here
```

# Stellar & Rust



Jão is learning Rust:

## 4) Jão lends his recipe book without transferring ownership

- Instead of moving ownership, you can borrow a value via a reference.
- An immutable reference &T allows reading.
- A mutable reference &mut T allows modification.
- The rule: either one mutable reference OR any number of immutable references — never both at once.

Rust code:

```
fn main() {  
    let mut stock = String::from("100 Baguncinhas");  
  
    // Multiple immutable borrows - all valid simultaneously  
    let r1 = &stock;  
    let r2 = &stock;  
    println!("r1={r1}, r2={r2}");  
  
    // Mutable borrow - exclusive, no other refs allowed  
    let r3 = &mut stock;  
    r3.push_str(" + 50 restocked");  
    println!("Updated: {r3}");  
  
    // Borrow in function - no ownership transfer  
    let recipe = String::from("secret sauce");  
    let len = get_len(&recipe); // borrow, not move  
    println!("'{recipe}' has {len} chars"); // still valid!  
  
    // Mutable borrow passed to function  
    let mut order = String::from("Classic");  
    add_toppings(&mut order);  
    println!("Order: {order}"); // "Classic + mustard"  
}  
  
fn get_len(s: &String) -> usize {  
    s.len() // s is borrowed, not dropped  
}  
  
fn add_toppings(order: &mut String) {  
    order.push_str(" + mustard");  
}
```

# Stellar & Rust



João is learning Rust:

## 5) João defines a Baguncinha struct with data and behaviour

- A **struct** groups related fields.
- An **impl** block attaches methods.
- Methods taking **&self** are read-only; those taking **&mut self** can modify the struct.
- Associated functions (no **self**) act as constructors.

Rust code:

```
#[allow(dead_code)]
#[derive(Debug)]
struct Baguncinha {
    name: String,
    price: f64,
    stock: u32,
    vegan: bool,
}

impl Baguncinha {
    // Constructor (no self - associated function)
    fn new(name: &str, price: f64, stock: u32) -> Self {
        Baguncinha { name: name.to_string(), price, stock, vegan: false }
    }

    // Immutable method - reads fields
    fn total_value(&self) -> f64 {
        self.stock as f64 * self.price
    }

    // Mutable method - modifies state
    fn sell(&mut self, qty: u32) -> Result<f64, &str> {
        if qty > self.stock {
            return Err("Not enough stock");
        }
        self.stock -= qty;
        Ok(qty as f64 * self.price)
    }
}

fn main() {
    let mut classic = Baguncinha::new("Classic", 2.50, 100);
    println!("Value: ${:.2}", classic.total_value());
    match classic.sell(3) {
        Ok(rev) => println!("Revenue: ${rev:.2}"),
        Err(e)  => println!("Error: {e}"),
    }
    println!("{:?}", classic);
}
```

# Stellar & Rust



João is learning Rust:

## 6) João models order status and menu items with powerful Rust enums

- **enum** in Rust is far more powerful than in other languages — each variant can hold different data.
- Combined with **match**, enums force you to handle every possible case.
- **Option<T>** replaces null and **Result<T,E>** replaces exceptions.

Rust code:

```
[allow(dead_code)]
#[derive(Debug)]
enum MenuItem {
    Classic,
    ChiliBagunca { spice_level: u8 },
    CheeseBagunca { cheese_type: String },
    Combo(String, u32),
}

fn describe(item: &MenuItem) -> String {
    match item {
        MenuItem::Classic => "Classic Baguncinha".to_string(),
        MenuItem::ChiliBagunca { spice_level } =>
            format!("Chili (spice: {})", spice_level),
        MenuItem::CheeseBagunca { cheese_type } =>
            format!("Cheese with {cheese_type}"),
        MenuItem::Combo(name, qty) =>
            format!("{qty}x {name}"),
    }
}

fn find_item(name: &str) -> Option<MenuItem> {
    match name {
        "classic" => Some(MenuItem::Classic),
        "chili"   => Some(MenuItem::ChiliBagunca { spice_level: 3 }),
        _         => None,
    }
}

fn main() {
    let order = MenuItem::CheeseBagunca { cheese_type: "cheddar".to_string() };
    println!("{}", describe(&order));
    // if let: handle one variant
    if let Some(found) = find_item("chili") {
        println!("Found: {}", describe(&found));
    }
    // unwrap_or: default value for None
    let item = find_item("pizza").unwrap_or(MenuItem::Classic);
    println!("Fallback: {}", describe(&item));
}
```



# Stellar & Rust



João is learning Rust:

## 7) João defines shared behaviour for all menu items with traits

- A **trait** defines shared behaviour — like an interface.
- Any type can implement a trait.
- Generic functions with trait bounds work on any type implementing that trait.
- This is Rust's zero-cost abstraction: all dispatch is resolved at compile time.

Rust code:

```
#[allow(dead_code)]
trait Priceable {
    fn price(&self) -> f64;
    // Default implementation - can override
    fn discount(&self, pct: f64) -> f64 {
        self.price() * (1.0 - pct / 100.0)
    }
}

#[derive(Debug, Clone)]
struct Bagunca { name: String, base_price: f64 }
#[derive(Debug, Clone)]
struct Drink { size_ml: u32, base_price: f64 }

impl Priceable for Bagunca {
    fn price(&self) -> f64 { self.base_price }
}

impl Priceable for Drink {
    fn price(&self) -> f64 {
        self.base_price * (self.size_ml as f64 / 250.0)
    }
}

// Generic: works for any T that implements Priceable
fn total<T: Priceable>(items: &[T]) -> f64 {
    items.iter().map(|x| x.price()).sum()
}

fn main() {
    let baguncinhas = vec![
        Bagunca { name: "Classic".into(), base_price: 2.50 },
        Bagunca { name: "Chili".into(), base_price: 3.00 },
    ];
    println!("Total: ${:.2}", total(&baguncinhas)); // $5.50
    println!("Discount: ${:.2}", baguncinhas[0].discount(10.0)); // $2.25
}
```

# Stellar & Rust



João is learning Rust:

## 8) João manages daily orders with Rust standard collections

- Rust's two main heap-allocated collections are ***Vec<T>*** (dynamic array) and ***HashMap<K,V>*** (hash map).
- The iterator system lets you chain ***.filter()***, ***.map()***, ***.sum()*** lazily — nothing computes until consumed.

Rust code:

```
#[allow(dead_code)]
use std::collections::HashMap;

#[derive(Debug)]
struct Order { customer: String, item: String, paid: f64 }

fn main() {
    let mut orders: Vec<Order> = Vec::new();
    orders.push(Order { customer: "Alice".into(), item: "Classic".into(), paid: 5.00 });
    orders.push(Order { customer: "João Jr".into(), item: "Chili".into(), paid: 3.00 });
    orders.push(Order { customer: "Alice".into(), item: "Drink".into(), paid: 2.00 });

    // Iterator chain: total revenue
    let total: f64 = orders.iter().map(|o| o.paid).sum();
    println!("Revenue: ${total:.2}");

    // Filter: Alice's orders only
    let alice: Vec<Order> = orders.iter()
        .filter(|o| o.customer == "Alice")
        .collect();
    println!("Alice: {} orders", alice.len()); // 2

    // HashMap: revenue per customer
    let mut rev_map: HashMap<String, f64> = HashMap::new();
    for o in &orders {
        *rev_map.entry(o.customer.clone()).or_insert(0.0) += o.paid;
    }
    for (cust, rev) in &rev_map {
        println!("{cust}: ${rev:.2}");
    }
}
```

# Stellar & Rust



João is learning Rust:

## 9) João handles payment failures gracefully —

Rust has no exceptions

- Rust has no exceptions.
- Errors are returned as values via ***Result<T,E>***.
- The ***?*** operator propagates errors automatically: if ***Err***, return it; if ***Ok***, unwrap the value.
- Custom error types give callers structured information.

Rust code:

```
use std::fmt;
#[derive(Debug)]
enum StandError { OutOfStock { item: String },
                  InvalidQty { requested: u32 },
                  PaymentShort { expected: f64, got: f64 }, }

impl fmt::Display for StandError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            Self::OutOfStock { item } => write!(f, "{item} is sold out"),
            Self::InvalidQty { requested } => write!(f, "Invalid qty: {requested}"),
            Self::PaymentShort { expected, got } =>
                write!(f, "Need ${expected:.2}, got ${got:.2}"),
        }
    }
}

impl std::error::Error for StandError {}

struct Stand { stock: u32 }

impl Stand {
    fn sell(&mut self, qty: u32, pay: f64) -> Result<f64, StandError> {
        if qty == 0 { return Err(StandError::InvalidQty { requested: qty }); }
        if qty > self.stock { return Err(StandError::OutOfStock { item:
            "Baguncinha".into() }); }
        let due = qty as f64 * 2.50;
        if pay < due { return Err(StandError::PaymentShort { expected: due, got: pay }); }
        self.stock -= qty;
        Ok(pay - due)
    } // change
}

fn run(stand: &mut Stand) -> Result<(), StandError> {
    let change = stand.sell(2, 5.00)?; // ? propagates Err
    println!("Change: ${change:.2}");
    Ok(())
}

fn main() {
    let mut stand = Stand { stock: 5 };
    match run(&mut stand) {
        Ok(()) => println!("Done!"),
        Err(e) => println!("Error: {e}"),
    }
}
```

# Stellar & Rust



João is learning Rust:

## 10) João processes daily reports with closures and lazy iterator chains

- A closure is an anonymous function that captures variables from its scope.
- Iterator methods take closures and form lazy chains — nothing executes until consumed.
- The compiler inlines everything: zero overhead.

Rust code:

```
fn main() {  
    let sales: Vec<(&str, u32, f64)> = vec![  
        ("Classic", 5, 12.50),  
        ("Chili", 3, 9.00),  
        ("Veggie", 2, 5.50),  
        ("Chili", 1, 3.00),  
    ];  
    // Closure stored in a variable  
    let is_popular = |qty: u32| qty >= 3;  
    // Chain: filter -> map -> collect  
    let popular: Vec<&str> = sales.iter()  
        .filter(|(_, qty, _)| is_popular(*qty))  
        .map(|(name, _, _)| *name)  
        .collect();  
    println!("Popular: {popular:?}");  
  
    // fold: compute total revenue  
    let total = sales.iter()  
        .fold(0.0f64, |acc, (_, _, rev)| acc + rev);  
    println!("Total: ${total:.2}");  
  
    // move closure: take ownership of captured var  
    let threshold = 5.0f64;  
    let big: Vec<_> = sales.iter()  
        .filter(move |(_, _, rev)| *rev > threshold)  
        .collect();  
    println!("Above ${threshold}: {} sales", big.len());  
}
```

# Stellar & Rust



João is learning Rust:

- In order to try Soroban, João uses <http://soropg.com> (Soroban Playground):
  - It is a browser based IDE (Integrated Development Environment)
  - Used to write, compile and test Rust Smart Contracts.
  - WASM is generated on cloud.

The screenshot shows the Soroban Playground IDE interface. The main editor displays a Rust file named `src/lib.rs` with the following code:

```
1  #![no_std]
2
3  use soroban_sdk::{contract, contractimpl, Env};
4
5  #[contract]
6  pub struct ExampleContract;
7
8  #[contractimpl]
9  impl ExampleContract {
10     pub fn add(_env: Env, a: i32, b: i32) -> i32 {
11         a + b
12     }
13 }
14
15 #[cfg(test)]
16 mod test;
17
```

The interface includes a sidebar on the left with icons for CREATE, TEST, BUILD, DEPLOY, and EXPLORE. The bottom panel shows a file explorer with a workspace containing `src`, `lib.rs`, `test.rs`, and `Cargo.toml`.

# Stellar & Rust & Contracts



João is learning Rust & Contracts:

- <http://soropg.com> steps:
  - Include smart contract code in **lib.rs** file
  - Include **test.rs** code → run “locally”
  - **Build** → generate WASM file
  - **Deploy** → “upload” contract on Stellar network
    - **Generate** or Connect Wallet
  - **Explore** → Load → run contract

The screenshot shows the SoroPG IDE interface. The main editor displays the `src/lib.rs` file with the following Rust code:

```
1  #![no_std]
2
3  use soroban_sdk::{contract, contractimpl, Env};
4
5  #[contract]
6  pub struct ExampleContract;
7
8  #[contractimpl]
9  impl ExampleContract {
10     pub fn add(_env: Env, a: i32, b: i32) -> i32 {
11         a + b
12     }
13 }
14
15 #[cfg(test)]
16 mod test;
17
```

The interface includes a sidebar on the left with icons for CREATE, TEST, BUILD, DEPLOY, and EXPLORE. The bottom panel shows a file explorer with a workspace containing `src`, `lib.rs`, `test.rs`, and `Cargo.toml`.

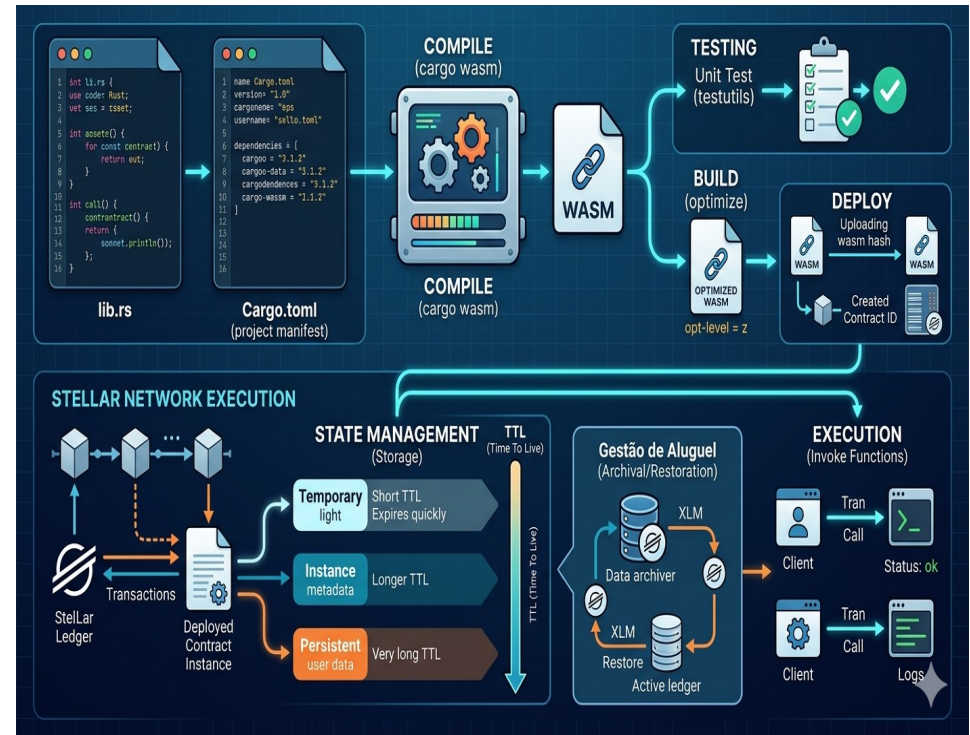
# Stellar & Rust & Contracts



João is learning Rust & Contracts:

- Stellar smart contract **life cycle**:

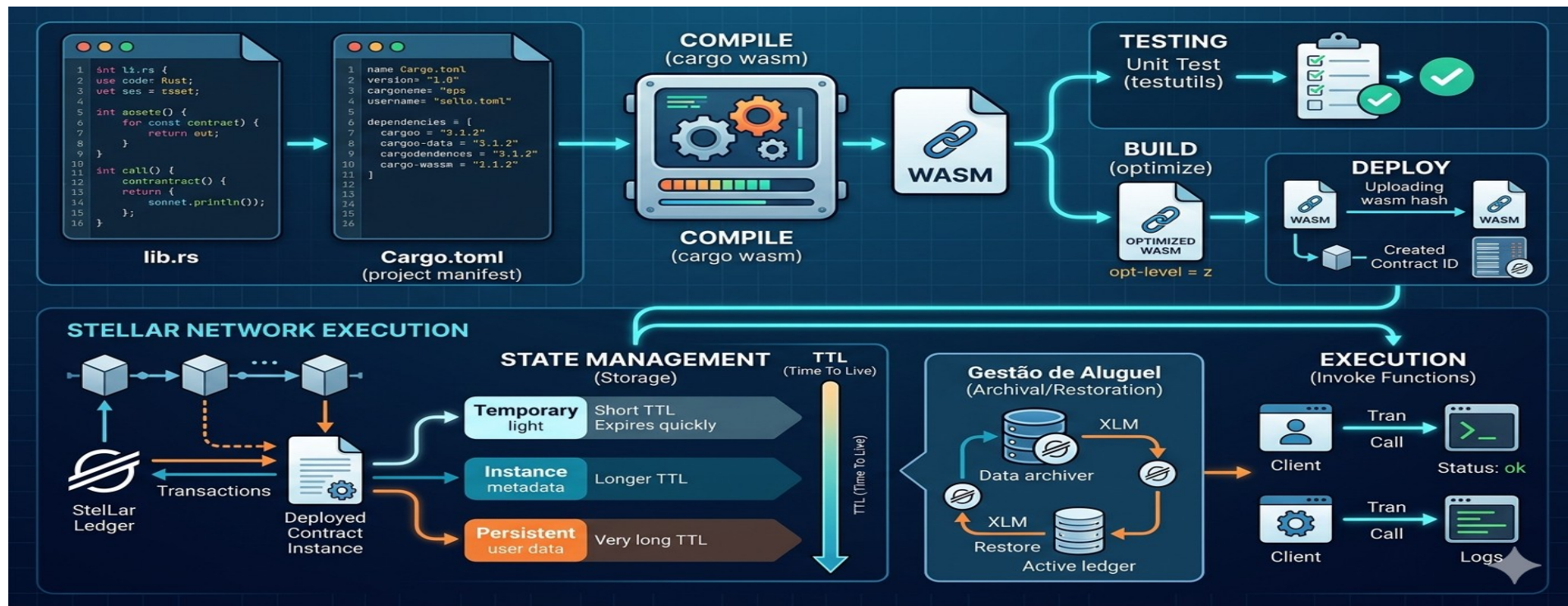
- Include soroban in code
- Build → generate **wasm** file
- Test → all contract's behavior are tested
- Deploy → “upload” wasm on Stellar network
  - Contract sign → **<contract id>**
  - Persistence mode → TTL (Time To Live)
- Invoke contract methods → **transactions**
- Reactivate



# Stellar & Rust & Contracts



João is learning Rust & Contracts:





# Stellar & Rust



Jão is learning Rust:

## 11) Jão applies everything: Rust knowledge meets Soroban smart contracts

- Now Jão applies every Rust concept to write real Soroban contracts.
- The **soroban\_sdk** crate brings Rust to the blockchain.
- Key differences from std Rust: **no\_std** environment (no OS, no threads), XDR serialisation for all stored types, and deterministic execution — same input must produce same result on every node.

Rust code :

```
#![no_std]
use soroban_sdk::{contract, contractimpl, contracttype,
                  Address, Env, String, symbol_short};

// Rust struct from Part 1 → now #[contracttype]
#[contracttype]
#[derive(Clone)]
pub struct Config {
    pub name: String, // soroban String, not std
    pub price: i128,  // BAGU token units (7 decimals)
}

// Rust enum from Part 1 → typed storage keys
#[contracttype]
pub enum DataKey {
    Admin,
    Config,
    Stock,
    TotalSold,
    Order(u32),
}
```

# Stellar & Rust



Jão is learning Rust:

## 11) Jão applies everything: Rust knowledge meets Soroban smart contracts

Rust code (continuation) :

```
#[contract]
pub struct Baguncinhandstand; // stateless - state lives in storage
#[contractimpl]
impl Baguncinhandstand {
    pub fn init(env: Env, admin: Address) {
        env.storage().instance().set(&DataKey::Admin, &admin);
        env.storage().instance().extend_ttl(1000, 17280);
    }
}

#[contractimpl]
impl Baguncinhandstand {
    // INSTANCE: small global config - always loaded
    pub fn set_config(env: Env, caller: Address, cfg: Config) {
        caller.require_auth();
        env.storage().instance().set(&DataKey::Config, &cfg);
        env.storage().instance().extend_ttl(1000, 17280);
    }
}
```

Rust code (continuation) :

```
// PERSISTENT: per-order data, independent TTL
pub fn place_order(env: Env, customer: Address, qty: u32) -> u32 {
    customer.require_auth();

    let id: u32 = env.storage().persistent()
        .get(&DataKey::TotalSold).unwrap_or(0u32) + 1;
    env.storage().persistent()
        .set(&DataKey::Order(id), &(customer, qty));
    env.storage().persistent()
        .set(&DataKey::TotalSold, &id);
    // ALWAYS extend_ttl after write
    env.storage().persistent()
        .extend_ttl(&DataKey::TotalSold, 17280, 120960);
    Id
}

// TEMPORARY: nonce - deleted in ~80s, prevents replay
pub fn pay(env: Env, customer: Address, nonce: u64) {
    customer.require_auth();
    let key = (symbol_short!("nonce"), nonce);
    assert!(env.storage().temporary().has(&key), "Replay attack");
    env.storage().temporary().set(&key, &true);
    env.storage().temporary().extend_ttl(&key, 100, 100);
}

#[cfg(test)]
mod test;
```

# Stellar & Rust



João is learning Rust:

## 11) João applies Unit Tests before deploy in order to prevent problems on contracts.

Rust code :

```
#![cfg(test)]
use super::*; // Importa tudo que definimos no lib.rs
use soroban_sdk::{testutils::Address as _, Address, Env, String};
#[test]
fn test_baguncinha_lifecycle() {
    let env = Env::default();
    env.mock_all_auths();
    let contract_id = env.register(BaguncinhaStand, ());
    let client = BaguncinhaStandClient::new(&env, &contract_id);
    let admin = Address::generate(&env);
    let customer = Address::generate(&env);
    client.init(&admin);
    let config_name = String::from_str(&env, "X-Bagunca");
    let lanche_config = Config {
        name: config_name,
        price: 25_0000000, // Ex: 25.0000000 unidades (7 decimais)
    };
};
```

Rust code (continuation) :

```
...
    client.set_config(&admin, &lanche_config);
    let order_id_1 = client.place_order(&customer, &2);
    let order_id_2 = client.place_order(&customer, &1);
    assert_eq!(order_id_1, 1);
    assert_eq!(order_id_2, 2);
    let nonce: u64 = 12345;
    client.pay(&customer, &nonce); }

#[test]
#[should_panic(expected = "Replay attack")]
fn test_replay_attack_prevention() {
    let env = Env::default();
    env.mock_all_auths();
    let contract_id = env.register(BaguncinhaStand, ());
    let client = BaguncinhaStandClient::new(&env, &contract_id);
    let customer = Address::generate(&env);
    let nonce: u64 = 999;
    client.pay(&customer, &nonce);
    client.pay(&customer, &nonce); // Isso deve causar panic
}
```

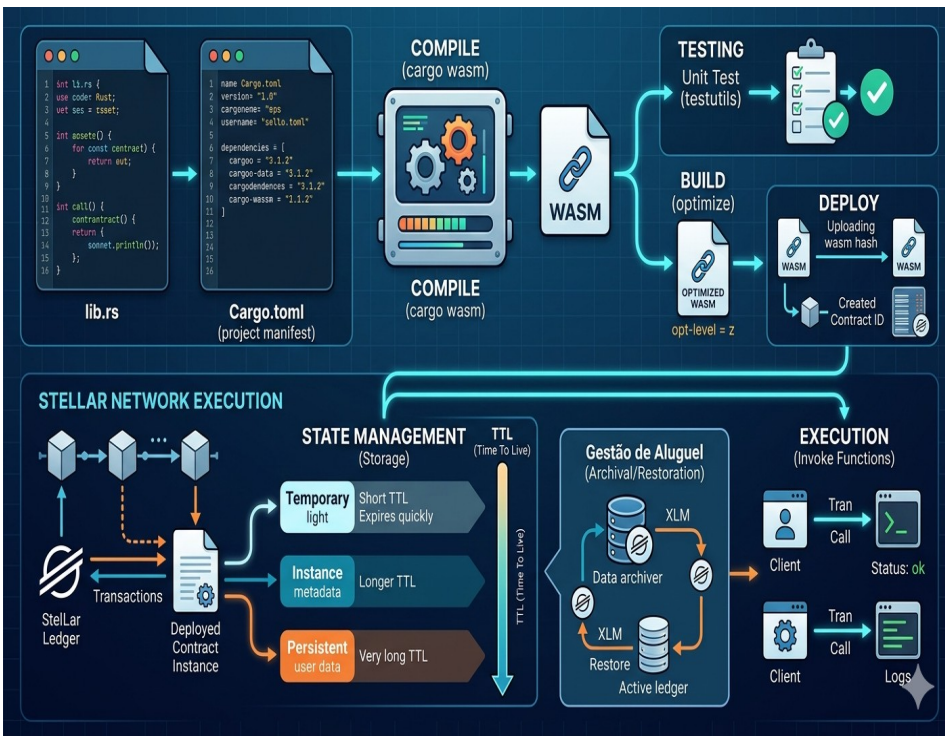
# Stellar & Rust & Contracts



Jão is learning Rust & Contracts:

- Stellar smart contract life cycle:

- ...
- Deploy → “upload” wasm on Stellar network
  - Contract sign → **<contract id>**
  - Block Explorer → Stellar Expert UI
- Explorer → Load Contract
  - Invoke contract methods → **transactions**
  - init()** → **admin** → **<public key>**
  - place\_order()** → should import secret key
  - pay()** → use XLM token\_address →  
CDLZFC3SYJYDZT7K67VZ75HPJVIEUVNIXF47ZG  
2FB2RMQVU2HHGCYSC



# Stellar & Rust & Contracts



Jão is learning Rust & Contracts:

- Stellar smart contract life cycle:

- ...

- Deploy → “upload” wasm on Stellar network

- Contract sign → **<contract id>**

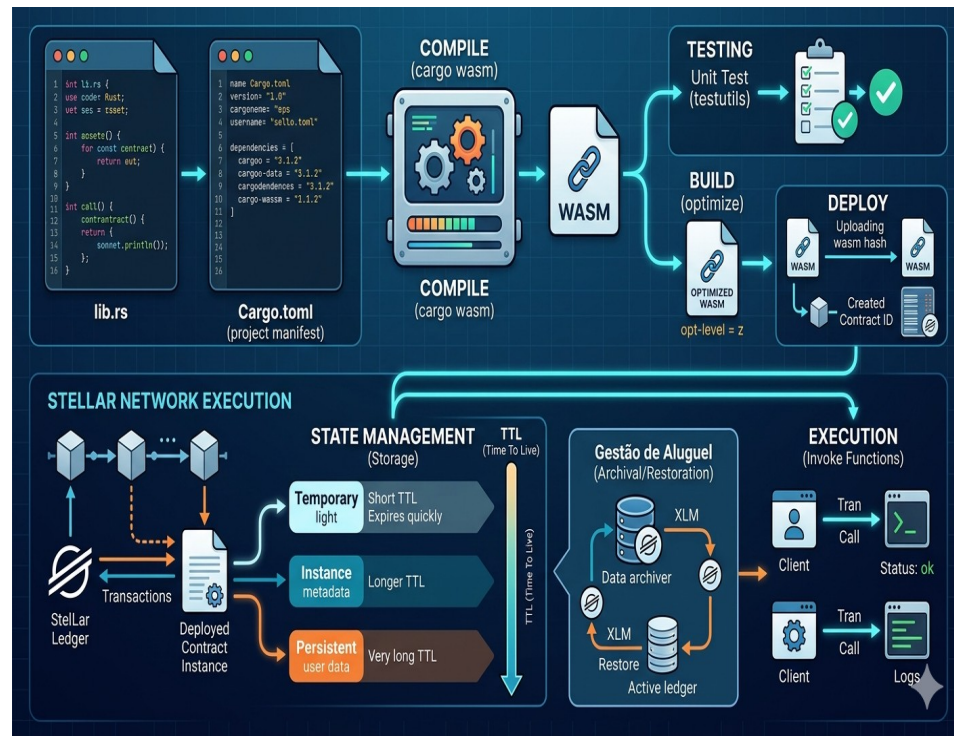
- Block Explorer → Stellar Expert UI

- Explorer → Load Contract

- Invoke contract methods → **transactions**

- Init** → **admin** → **<public key>**

- place\_order** → should import secret key



# Stellar & Rust



João is learning Rust:

## 12) João maps Rust variables to on-chain storage

- In Part 1 João used *let* and *let mut*.
- In Soroban all state lives on the ledger — no in-memory globals between calls.
- Instance is like a global const (loaded every call).
- Persistent is like a long-lived *let mut* (independent TTL, archivable).
- Temporary is like a local variable (deleted on expiry).

Rust code:

```
...  
#[contractimpl]  
impl Baguncinhastand {  
    pub fn set_config(env: Env, caller: Address, cfg: Config) {  
        caller.require_auth();  
        env.storage().instance().set(&DataKey::Config, &cfg);  
        env.storage().instance().extend_ttl(1000, 17280);  
    }  
    pub fn place_order(env: Env, customer: Address, qty: u32) -> u32 {  
        customer.require_auth();  
        let id:u32=env.storage().persistent()  
            .get(&DataKey::TotalSold).unwrap_or(0u32)+1;  
        env.storage().persistent().set(&DataKey::Order(id), &(customer, qty));  
        env.storage().persistent().set(&DataKey::TotalSold, &id);  
        env.storage().persistent().extend_ttl(&DataKey::TotalSold, 17280, 120960);  
        id  
    }  
    pub fn pay(env: Env, customer: Address, nonce: u64, token_address: Address,  
amount: il28) {  
        customer.require_auth();  
        let key = (symbol_short!("nonce"), nonce);  
        assert!(!env.storage().temporary().has(&key), "Replay attack");  
        env.storage().temporary().set(&key, &true);  
        env.storage().temporary().extend_ttl(&key, 100, 100);  
        let admin: Address = env.storage().instance()  
            .get(&DataKey::Admin).expect("Not initialized");  
        let token_client = soroban_sdk::token::Client::new(&env, &token_address);  
        token_client.transfer(&customer, &admin, &amount);  
    }  
}
```

# Stellar & Rust



João is learning Rust:

## 13) João applies Rust structs and enums to build role-based access control

- The **require\_auth()** call uses Rust's Address type. Role checking combines struct methods and enum matching from Part 1.
- The pattern is always: authenticate (who signed?) then authorise (are they allowed?).
- 

Rust code:

```
#[contracttype]
#[derive(Clone, PartialEq)]
pub enum Role { Admin, Manager, Cashier }
#[contractimpl]
impl BaguncinhaStand {
    pub fn assign_role(
        env: Env, caller: Address, staff: Address, role: Role
    ) {
        caller.require_auth(); // authenticate
        let admin: Address = env.storage().instance()
            .get(&DataKey::Admin).unwrap();
        assert_eq!(caller, admin, "Only admin"); // authorise
        env.storage().persistent().set(&staff, &role);
    }
    pub fn restock(env: Env, caller: Address, qty: u32) {
        caller.require_auth();
        let role: Role = env.storage().persistent()
            .get(&caller).unwrap_or(Role::Cashier);
        match role {
            Role::Admin | Role::Manager => {} // allowed
            Role::Cashier => panic!("Not authorised to restock"),
        }
        let stock: u32 = env.storage().persistent()
            .get(&DataKey::Stock).unwrap_or(0);
        env.storage().persistent()
            .set(&DataKey::Stock, &(stock + qty));
    }
}
```

# Stellar & Rust



João is learning Rust:

## 14) João combines Rust enums and Result into contract events and structured errors

- Soroban errors are Rust enums with **`#[contracterror]`** — the same `StandError` from Part 1, but each variant gets an integer code in the transaction XDR.
- Events use **`env.events().publish(topics, data)`** — all Rust types that implement **`Into<ScVal>`**.

Rust code:

```
...
#[contracterror]
#[derive(Clone, Copy, Debug, PartialEq)]
pub enum StandError {
    OutOfStock = 201,
    InsufficientStock = 202,
    InvalidQuantity = 203,
    PaymentShort = 301,
}

#[contractimpl]
impl BaguncinHastand {
    ...
    pub fn buy(env: Env, customer: Address, qty: u32, payment: i128,
    ) -> Result<u32, StandError> {
        customer.require_auth();
        if qty == 0 { return Err(StandError::InvalidQuantity); }
        let stock: u32 = env.storage().persistent().get(&DataKey::Stock)
            .unwrap_or(0);
        if stock == 0 { return Err(StandError::OutOfStock); }
        if qty > stock { return Err(StandError::InsufficientStock); }
        let price: i128 = (qty as i128) * 100_000_000;
        if payment < price { return Err(StandError::PaymentShort); }
        env.storage().persistent().set(&DataKey::Stock, &(stock - qty));
        env.events().publish(
            (symbol_short!("sell"), customer.clone()),
            SaleEvent {customer, qty, revenue: price, }, );
        Ok(1u32)
    }
}
```



# Stellar & Rust



João is learning Rust:

## 15) João uses Rust traits to interact with the BAGU token via TokenClient

- ***TokenClient*** from ***soroban\_sdk::token*** implements the SEP-41 token trait.
- This is the exact same trait concept from Part 1:
  - ***impl Priceable for Bagunca*** is structurally identical to
  - ***impl TokenInterface for StellarAssetContract***.
- Calling ***token.transfer()*** is calling a trait method.

Rust code:

```
use soroban_sdk::token;
#[contractimpl]
impl BaguncinhaStand {
    pub fn buy_with_hot(
        env: Env, customer: Address, qty: u32,
        bagu_token: Address, // SAC contract address
    ) -> Result<u32, StandError> {
        customer.require_auth();
        let stock: u32 = env.storage().persistent()
            .get(&DataKey::Stock).unwrap_or(0);
        if qty > stock {
            return Err(StandError::InsufficientStock);
        }
        // TokenClient uses the SEP-41 trait - same as impl Trait
        let token = token::TokenClient::new(&env, &bagu_token);
        let price: i128 = (qty as i128) * 100_000_000;
        if token.balance(&customer) < price {
            return Err(StandError::PaymentShort);
        }
        // transfer() is a trait method call
        token.transfer(&customer, &env.current_contract_address(), &price);
        env.storage().persistent().set(&DataKey::Stock, &(stock - qty));
        env.storage().persistent().extend_ttl(&DataKey::Stock, 17280, 120960);
        Ok(1u32)
    }

    pub fn withdraw(env: Env, caller: Address, hot: Address) {
        caller.require_auth();
        let admin: Address = env.storage().instance().get(&DataKey::Admin).unwrap();
        assert_eq!(caller, admin);
        let token = token::TokenClient::new(&env, &hot);
        let bal = token.balance(&env.current_contract_address());
        if bal > 0 {token.transfer(&env.current_contract_address(), &admin, &bal);}
    }
}
```

# Class 3 – Smart Contracts with Soroban

Before coffee:

## Rust ultrafast journey

- Baguncinha transactions
- 

After coffee:

## Smart contracts in Soroban

- Smart token
- Claimable clawback

# Stellar & Soroban



Jão is using Soroban:

1) Jão is convinced to put the entire Baguncinha on a Soroban smart contract.

- Day 1 of the blockchain era.
- Jão is skeptical but agrees if he can keep his secret sauce recipe private.
- A Soroban contract is a Rust struct decorated with `#[contract]`, with its functions exposed via `#[contractimpl]`.
- The Env is Jão's "counter" — it gives access to storage, events, authentication, and cryptographic functions.

Rust code :

```
#![no_std]
use soroban_sdk::{contract, contractimpl, contracttype,
                  symbol_short, Env, Symbol, String, Address};
// The Baguncinha contract struct (stateless - state lives in storage)
#[contract]
pub struct Baguncinha;
// Custom type stored in contract - derives ContractType for XDR serialization
#[contracttype]
pub enum DataKey {
    Admin, StandName, BaguncinhasLeft, TotalSold, Order(u32), LastSaleNonce,
}

#[contractimpl]
impl Baguncinha {
    // Initialize: set admin and stand name
    pub fn init(env: Env, admin: Address, name: String) {
        assert!(!env.storage().instance().has(&DataKey::Admin),
            "Already initialized");
        env.storage().instance().set(&DataKey::Admin, &admin);
        env.storage().instance().set(&DataKey::StandName, &name);
        env.storage().persistent().set(&DataKey::BaguncinhasLeft, &0u32);
        env.storage().persistent().set(&DataKey::TotalSold, &0u32);
    }
    // Get the stand name (no auth required - public read)
    pub fn get_name(env: Env) -> String {
        env.storage().instance()
            .get(&DataKey::StandName)
            .unwrap()
    }
}
```

# Stellar & Soroban



Jão is using Soroban:

2) Instance storage: contract-wide config that lives as long as the contract.

- Jão's stand has a name, an admin address, and a version number.
- These never change unless Jão explicitly updates them — they're the stand's identity. Instance storage is the right tool: it's loaded on every invocation (fast reads), its **TTL** is tied to the contract instance itself, and it's ideal for small, frequently-read config.
- The caveat: all instance storage is loaded together, so never put growing data here (like a list of all orders — that belongs in persistent storage by key).

Rust code :

```
#[contractimpl]
impl Baguncinha {
    /// Read-only getter — no TTL extension needed for reads
    pub fn get_admin(env: Env) -> Address {
        env.storage().instance()
            .get(&DataKey::Admin)
            .unwrap_or_else(|| panic!("Not initialized"))    }

    /// Mutating function — extend TTL on every write to keep alive
    pub fn set_name(env: Env, caller: Address, new_name: String) {
        // Verify caller is admin (auth covered in Chapter 3)
        caller.require_auth();
        let admin: Address = env.storage().instance()
            .get(&DataKey::Admin).unwrap();
        assert_eq!(caller, admin, "Only admin can rename");
        env.storage().instance().set(&DataKey::StandName, &new_name);
        // 💡 Extend TTL: min 1000 ledgers, max 17280 (~24h)
        // If current TTL > min, no-op (won't reset unnecessarily)
        env.storage().instance().extend_ttl(1000, 17280);    }

    /// Pattern: always extend in any mutating function
    pub fn set_version(env: Env, caller: Address, v: u32) {
        caller.require_auth();
        env.storage().instance().set(
            &symbol_short!("version"), &v    );
        env.storage().instance().extend_ttl(1000, 17280);    }
```

# Stellar & Soroban



Jão is using Soroban:

## 3) Instance storage: contract-wide config that lives as long as the contract.

- Every baguncinhaorder Jão receives is stored persistently — keyed by order ID.
- Each key has an independent TTL.
- If Jão's stand sits idle for months, the order history could get archived (moved off-chain) to save ledger space.
- Jão extends the TTL whenever an order is accessed. If an order gets archived (TTL hits zero), it can be restored via RestoreFootprintOp — it's not lost forever.
- This is Soroban's state rent model: pay for what you use.

Rust code :

```
#[contracttype]
pub struct Order {
    pub customer: Address, pub item: Symbol, pub quantity: u32,
    pub paid: i128, pub filled: bool,}
const ORDER_TTL_MIN: u32 = 17280;    // ~24 hours
const ORDER_TTL_MAX: u32 = 120960;  // ~7 days
#[contractimpl]
impl Baguncinha {
    pub fn place_order( env: Env, customer: Address, item: Symbol,
                        quantity: u32, payment: i128, ) -> u32 {
        customer.require_auth();
        // Get and increment order counter from persistent storage
        let order_id: u32 = env.storage().persistent()
            .get(&DataKey::TotalSold).unwrap_or(0u32) + 1;
        let order = Order {
            customer: customer.clone(),
            item, quantity, paid: payment, filled: false, };
        // Store order with key Order(id) — each has its own TTL
        env.storage().persistent().set(&DataKey::Order(order_id), &order);
        env.storage().persistent()
            .extend_ttl(&DataKey::Order(order_id), ORDER_TTL_MIN, ORDER_TTL_MAX);
        // Update total counter
        env.storage().persistent().set(&DataKey::TotalSold, &order_id);
        env.storage().persistent().extend_ttl(&DataKey::TotalSold,
            ORDER_TTL_MIN, ORDER_TTL_MAX);
        order_id    }    }
```

# Stellar & Soroban



Jão is using Soroban:

## 4) Temporary storage: anti-replay nonces and session-scoped data

- Jão's payment system is at risk of replay attacks: an attacker capturing a payment transaction could try to re-submit it later.
- Jão uses temporary storage to record a nonce (a unique payment ID) that expires after ~80 seconds.
- Temporary storage is deleted permanently when it expires (unlike persistent, which archives).
- This makes it perfect for: anti-replay nonces, price oracle data (valid for 1 minute), session tokens, and rate-limiting windows.

Rust code :

```
#[contracttype]
pub enum TempKey { PaymentNonce(u64), PriceCache, RateLimit(Address), }

#[contractimpl]
impl Baguncinha {
    /// Pay for an order with a nonce to prevent replay attacks
    pub fn pay_with_nonce( env: Env, customer: Address, order_id: u32, nonce: u64,
    ) {
        customer.require_auth();
        // Check nonce has not been used
        let key = TempKey::PaymentNonce(nonce);
        Assert!( !env.storage().temporary().has(&key),
            "Nonce already used - replay attack detected!" );

        // Mark nonce as used (TTL: 100 ledgers ≈ 8 minutes)
        env.storage().temporary().set(&key, &true);
        env.storage().temporary().extend_ttl(&key, 100, 100);

        // Rate limit: max 5 orders per address per window
        let rate_key = TempKey::RateLimit(customer.clone());
        let count: u32 = env.storage().temporary().get(&rate_key).unwrap_or(0u32);
        assert!(count < 5, "Rate limit exceeded - slow down!");
        env.storage().temporary().set(&rate_key, &(count + 1));
        env.storage().temporary().extend_ttl(&rate_key, 200, 200);    } }
```

# Stellar & Soroban



Jão is using Soroban:

## 5) Authentication: Jão controls who can restock

- Jão's stand has a role-based system: only the Admin can restock baguncinhas, only the Manager can set prices, and any customer can place orders. Soroban's **require\_auth()** is the gatekeeper.
- When a function calls **address.require\_auth()**, the runtime verifies that the address has actually signed the transaction (or approved this sub-call in a cross-contract scenario).
- It's atomic and unforgeable.

Rust code :

```
#[contracttype]
pub enum Role { Admin, Manager, Cashier }
#[contractimpl]
impl Baguncinha {
    /// Only Admin can restock inventory
    pub fn restock(env: Env, caller: Address, amount: u32) {
        caller.require_auth();
        let admin: Address = env.storage().instance()
            .get(&DataKey::Admin).unwrap();
        assert_eq!(caller, admin, "Only admin can restock");
        let current: u32 = env.storage().persistent()
            .get(&DataKey::BaguncinhasLeft).unwrap_or(0);
        env.storage().persistent()
            .set(&DataKey::BaguncinhasLeft, &(current + amount));
        env.storage().instance().extend_ttl(1000, 17280);
    }
    pub fn approve_order(env: Env, manager: Address, order_id: u32) {
        /// Auth scoped to this exact function+args combination
        manager.require_auth_for_args(
            ("approve_order", order_id).into_val(&env)
        );
        /// Manager can only approve the specific order_id they signed for
        /// Cannot be replayed for a different order_id
        /// Public - no auth needed (anyone can read)
    }
    pub fn get_stock(env: Env) -> u32 {
        env.storage().persistent()
            .get(&DataKey::BaguncinhasLeft).unwrap_or(0)
    }
}
```

# Stellar & Soroban



Jão is using Soroban:

## 6) Custom account contract: Jão's stand needs 2-of-3 approval

- For large restocking orders (over 500 baguncinhas), Jão requires 2 of 3 managers to approve. He deploys a custom account contract — a contract that replaces the default Ed25519 signature check with custom logic.
- The `__check_auth()` function is the single entry point the Soroban runtime calls to validate all auth for an address.
- This enables account abstraction: multi-sig policies, time locks, spending limits, social recovery.

Rust code :

```
#[contract]
pub struct MultiSigAccount;
#[contractimpl]
impl CustomAccountInterface for MultiSigAccount {
    type Signature = Vec<BytesN<64>>; // list of Ed25519 signatures
    type Error = Error;
    /// Called by runtime to validate: is this transaction authorized?
    fn __check_auth( env: Env, signature_payload: Hash<32>,
                    signatures: Vec<BytesN<64>>,auth_contexts: Vec<Context>,
    ) -> Result<(), Error> {
        let signers: Vec<BytesN<32>> =
            env.storage().instance().get(&symbol_short!("signers"))
                .ok_or(Error::NotInitialized)?;
        let threshold: u32 =
            env.storage().instance().get(&symbol_short!("thresh"))
                .unwrap_or(2); // default: 2-of-N
        let mut valid_sigs: u32 = 0;
        for sig in signatures.iter() {
            for signer in signers.iter() {
                if env.crypto().ed25519_verify(
                    &signer, &signature_payload.into(), &sig).is_ok() {
                    valid_sigs += 1;
                    Break; } } }
        if valid_sigs >= threshold {Ok()}
        } else { Err(Error::InsufficientSignatures) } }}
```



# Stellar & Soroban



Jão is using Soroban:

## 7) Events: Jão's dashboard listens to every baguncinhasold

- Jão's nephew built a live sales dashboard.
- Every time a baguncinha is sold or restocked, the contract emits an event.
- Off-chain apps subscribe to these events via the `getEvents` RPC method.
- Events have topics (indexed, filterable — like the event name and contract ID) and a data payload (arbitrary `ScVal`).
- They are stored for ~24 hours.
- Critically, events only exist if the transaction succeeds — they are part of the transaction result.

Rust code :

```
/// Event types for the Baguncinha
const TOPIC_SELL: Symbol = symbol_short!("sell");
const TOPIC_RESTOCK: Symbol = symbol_short!("restock");
const TOPIC_REFUND: Symbol = symbol_short!("refund");

#[contracttype]
pub struct SaleEvent { pub customer: Address, pub item: Symbol,
                      pub quantity: u32, pub total_paid: i128, pub order_id: u32, }

#[contractimpl]
impl Baguncinha {
    pub fn sell(env: Env, customer: Address, item: Symbol, qty: u32) -> u32 {
        customer.require_auth();
        // ... validate, deduct inventory, record order ...
        let order_id = 42u32; // from storage
        // Emit structured sell event
        // Topics: ("sell", item) -> filterable by dashboard
        // Data: full SaleEvent struct
        env.events().publish( ( TOPIC_SELL, item.clone(),
                               SaleEvent { customer: customer.clone(), item, quantity: qty,
                                             total_paid: (qty as i128) * 100_000_000, order_id,
                               } ),
                               order_id );
    }
}
```

# Stellar & Soroban



Jão is using Soroban:

## 8) Structured errors: Jão's contract tells callers exactly what went wrong

- When Alice tries to buy 10 baguncinhas but Jão only has 3 left, the contract should fail gracefully with a useful error code — not a cryptic panic.
- Soroban errors use a `#[contracterror]` enum where each variant maps to a unique integer.
- These codes appear in the transaction result XDR and can be parsed by clients.
- Functions return `Result<T, Error>` — the Rust way of handling failure.

Rust code :

```
#[contracterror]
#[derive(Clone, Copy, Debug)]
pub enum StandError {
    // Auth errors: 1xx
    NotAdmin = 101, NotAuthorized = 102,
    // Inventory errors: 2xx
    OutOfStock = 201, InsufficientStock = 202, InvalidQuantity = 203,
    // Payment errors: 3xx
    InsufficientPayment = 301, PaymentOverflow = 302,
    // Order errors: 4xx
    OrderNotFound = 401, OrderAlreadyFilled = 402, ReplayAttack = 403,
}

#[contractimpl]
impl Baguncinha {
    pub fn buy( env: Env, customer: Address, quantity: u32, payment: i128, ) ->
Result<u32, StandError> { customer.require_auth();
    if quantity == 0 {
        return Err(StandError::InvalidQuantity);
    }
    let stock: u32 = env.storage().persistent()
        .get(&DataKey::BaguncinhasLeft).unwrap_or(0);
    if stock == 0 { return Err(StandError::OutOfStock); }
    if quantity > stock { return Err(StandError::InsufficientStock); }
    // Price check (100 BAGU per baguncinha = 100 × 10^7 units)
    let price: i128 = (quantity as i128) * 100_000_000;
    if payment < price { return Err(StandError::InsufficientPayment); }
    env.storage().persistent().set(&DataKey::BaguncinhasLeft, &(stock -
quantity));
    Ok(42u32) } } // actual order_id from counter
```

# Stellar & Soroban



Jão is using Soroban:

## 9) Jão's contract accepts BAGU token payments via the Stellar Asset Contract

- Instead of tracking payments off-chain, Jão's contract directly holds and transfers BAGU tokens.
- When a customer buys a baguncinha, the contract pulls BAGU tokens from their account into the contract's own balance, then records the order.
- The Stellar Asset Contract (SAC) provides a TokenClient that any Soroban contract can use.
- The SAC wraps the classic BAGU asset in a SEP-41 interface, making the same token accessible from both classic Stellar operations and Soroban contracts.

Rust code :

```
use soroban_sdk::token;
#[contractimpl]
impl Baguncinha {
    pub fn buy_with_bagu(env: Env, customer: Address, quantity: u32,
        bagu_token: Address, // SAC contract address for BAGU
    ) -> Result<u32, StandError> { customer.require_auth();
        // Validate stock
        let stock: u32 = env.storage().persistent()
            .get(&DataKey::BaguncinhasLeft)
            .ok_or(StandError::OutOfStock)?;
        if quantity > stock { return Err(StandError::InsufficientStock); }
        // Calculate price: 100 BAGU per unit (1e7 = 1 BAGU)
        let price: i128 = (quantity as i128) * 100_000_000;
        // Pull BAGU tokens: customer -> this contract
        // transfer_from requires customer.require_auth() above
        let token = token::TokenClient::new(&env, &bagu_token);
        token.transfer(&customer, &env.current_contract_address(), &price, );
        env.storage().persistent()
            .set(&DataKey::BaguncinhasLeft, &(stock - quantity));
        env.events().publish( (TOPIC_SELL, symbol_short!("bagu_pay")),
            (customer, quantity, price), ); Ok(stock - quantity) }

    pub fn withdraw_proceeds( env: Env, caller: Address, bagu_token: Address,
    ) -> i128 { caller.require_auth();
        let admin: Address = env.storage().instance()
            .get(&DataKey::Admin).unwrap();
        assert_eq!(caller, admin, "Only admin can withdraw");
        let token = token::TokenClient::new(&env, &bagu_token);
        let balance = token.balance(&env.current_contract_address());
        if balance > 0 {token.transfer( &env.current_contract_address(), &admin, &balance, ); }
        Balance } }
```

# Stellar & Soroban



Jão is using Soroban:

## 10) Cross-contract: the stand calls the Loyalty Rewards contract automatically

- Jão's Baguncinha now calls a separate Loyalty Rewards contract on every sale.
- The architecture is modular: one contract for stand logic, one for loyalty points.
- They communicate via cross-contract calls.
- Soroban cross-contract calls are synchronous and atomic.
- If the loyalty contract fails, the entire transaction rolls back — both the sale and the reward.
- The calling contract can require\_auth checks that cascade through sub-calls.

Rust code :

```
// — loyalty_rewards.rs — SEPARATE CONTRACT —————
#[contract]
pub struct LoyaltyRewards;
#[contractimpl]
impl LoyaltyRewards {
    /// Award BAGU loyalty points — only callable by stand contract
    pub fn award_points( env: Env, stand: Address, customer: Address, amount: u32,) {
        // Only the stand contract can call this
        stand.require_auth();
        let authorized_stand: Address = env.storage().instance()
            .get(&symbol_short!("stand")).unwrap();
        assert_eq!(stand, authorized_stand);
        let current: u32 = env.storage().persistent()
            .get(&customer).unwrap_or(0u32);
        env.storage().persistent().set(&customer, &(current + amount)); }

// — baguncinha_stand.rs — CALLS THE LOYALTY CONTRACT —————
use loyalty_rewards::{LoyaltyRewards, LoyaltyRewardsClient};
#[contractimpl]
impl Baguncinha {
    pub fn buy_and_earn( env: Env, customer: Address, quantity: u32, loyalty_contract: Address,
    ) -> Result<u32, StandError> { customer.require_auth();
        let loyalty = LoyaltyRewardsClient::new(&env, &loyalty_contract);
        loyalty.award_points(
            &env.current_contract_address(), // stand authorizes itself
            &customer,
            &(quantity * 10), // 10 points per baguncinha
        );
        Ok(42u32)    }
```

# Stellar & Soroban



Jão is using Soroban:

## 11) Jão franchises his stand: a Deployer contract creates new stands on demand

- Jão's baguncinha empire is expanding!
- He creates a Franchise Factory: a single contract that deploys new Baguncinha contracts on demand for franchisees.
- The Deployer pattern lets a parent contract create child contracts with deterministic addresses.
- Every new stand gets a unique address derived from a salt (like the city name).
- Jão's factory keeps a registry of all deployed stands.
- Deployer contracts are the factory pattern of Soroban.

Rust code :

```
#[contract]
pub struct FranchiseFactory;

#[contractimpl]
impl FranchiseFactory {
    /// Deploy a new Baguncinha for a franchisee city
    pub fn open_franchise( env: Env, caller: Address, city: Symbol, franchisee: Address,
        stand_wasm_hash: BytesN<32>,
    ) -> Address { caller.require_auth();
        let factory_admin: Address = env.storage().instance()
            .get(&symbol_short!("admin")).unwrap();
        assert_eq!(caller, factory_admin);
        // Salt = city symbol -> deterministic address per city
        let salt: BytesN<32> = env.crypto().sha256(&city.to_xdr(&env));
        // Deploy the stand contract
        let stand_address= env.deployer().with_current_contract(salt).deploy_v2(stand_wasm_hash,());
        // Initialize the deployed stand
        let stand_client = BaguncinhaClient::new(&env, &stand_address);
        stand_client.init(&franchisee,
            &String::from_str(&env, &format!("{}", city)) );
        // Register in factory's persistent storage
        env.storage().persistent().set(&city, &stand_address);
        stand_address
    }

    /// Get stand address for a city
    pub fn get_stand(env: Env, city: Symbol) -> Option<Address> {
        env.storage().persistent().get(&city)
    }
}
```

# Stellar & Soroban



Jão is using Soroban:

## 12) Unit tests: Jão writes tests before deploying to mainnet

- Jão's nephew insists: no deployment without tests. Soroban has a built-in local testing environment that runs in native Rust — no network, no Docker, instant feedback.
- **Env::default()** creates an in-memory ledger. **env.mock\_all\_auths()** bypasses signature checks so you can test logic without real keypairs.
- You can fast-forward ledgers, check emitted events, and verify every storage mutation — all in a standard cargo test run.

Rust code :

```
#[cfg(test)]
mod tests {
    use super::*;
    use soroban_sdk::{testutils::{Address as _, Events}, Env};
    #[test]
    fn test_buy_success() {
        let env = Env::default();
        env.mock_all_auths(); // bypass signature verification
        let contract_id = env.register(Baguncinha, ());
        let client = BaguncinhaClient::new(&env, &contract_id);
        let admin = Address::generate(&env);
        let alice = Address::generate(&env);
        client.init(&admin, &String::from_str(&env, "Jão's Stand"));
        client.restock(&admin, &50u32);
        let order_id = client.buy(&alice, &2u32, &200_000_000i128).unwrap();
        assert_eq!(order_id, 1u32);
        assert_eq!(client.get_stock(), &48u32);
        let events = env.events().all();
        assert!(!events.is_empty(), "Expected sell event");
    }

    #[test]
    fn test_buy_out_of_stock() {
        let env = Env::default();
        env.mock_all_auths();
        let contract_id = env.register(Baguncinha, ());
        let client = BaguncinhaClient::new(&env, &contract_id);
        let admin = Address::generate(&env);
        let alice = Address::generate(&env);
        client.init(&admin, &String::from_str(&env, "Test"));
        let result = client.buy(&alice, &1u32, &100_000_000i128);
        assert_eq!(result, Err(StandError::OutOfStock));
    }
}
```

```
#[test]
fn test_time_window() {
    let env = Env::default();
    env.mock_all_auths();
    // Fast-forward the ledger close time
```

# Stellar & Soroban



Jão is using Soroban:

## 12) Unit tests: Jão writes tests before deploying to mainnet

- Jão's nephew insists: no deployment without tests. Soroban has a built-in local testing environment that runs in native Rust — no network, no Docker, instant feedback.
- **Env::default()** creates an in-memory ledger. **env.mock\_all\_auths()** bypasses signature checks so you can test logic without real keypairs.
- You can fast-forward ledgers, check emitted events, and verify every storage mutation — all in a standard cargo test run.

Rust code (continuation) :

```
...  
#[test]  
fn test_time_window() {  
    let env = Env::default();  
    env.mock_all_auths();  
    // Fast-forward the ledger close time  
    env.ledger().set(soroban_sdk::testutils::LedgerInfo {  
        timestamp: 1720441200, // Saturday 9AM  
        protocol_version: 22,  
        sequence_number: 1000,  
        network_id: Default::default(),  
        base_reserve: 10,  
        min_temp_entry_ttl: 16,  
        min_persistent_entry_ttl: 4096,  
        max_entry_ttl: 6312000,  
    });  
    // ... test time-dependent logic ...  
}
```

# Stellar & Soroban



Jão is using Soroban:

## 13) Jão's VIP membership card uses on-chain signature verification

- Jão launches a VIP Membership Card system.
- Each card is a signed credential: Jão's backend signs the customer's address + membership level with an Ed25519 key, and the contract verifies that signature on-chain before granting VIP prices.
- This is the meta-transaction / off-chain authorization pattern.
- The `env.crypto()` host functions provide native ed25519 verification, SHA-256 hashing, keccak256, and (Protocol 21+) secp256r1 (Passkey) for WebAuthn-based auth.

Rust code :

```
#[contracttype]
pub struct VipCredential {
    pub customer: Address, pub level: u32, pub valid_until: u64, }

#[contractimpl]
impl Baguncinha {
    /// Verify an offline-signed VIP credential and grant discount
    pub fn buy_vip( env: Env, customer: Address, quantity: u32, credential: VipCredential,
        signature: BytesN<64>,) -> Result<u32, StandError> { customer.require_auth();
        // Verify credential matches the caller
        assert_eq!(credential.customer, customer, "Credential not for this customer");
        // Check expiry against ledger timestamp
        let now = env.ledger().timestamp();
        assert!(now <= credential.valid_until, "VIP card expired");
        // Reconstruct the message Jão's backend signed
        let msg_bytes = credential.to_xdr(&env);
        let msg_hash = env.crypto().sha256(&msg_bytes);
        // Retrieve Jão's signing public key from instance storage
        let vip_signer: BytesN<32> = env.storage().instance()
            .get(&symbol_short!("vipsign")).unwrap();
        // Verify on-chain: Ed25519.verify(pubkey, hash, sig)
        env.crypto().ed25519_verify(&vip_signer, &msg_hash.into(), &signature, );
        // Panics if invalid - only Jão's backend can create valid VIP cards
        // Apply VIP discount (e.g. level 3 = 30% off)
        let discount = credential.level * 10; // 10% per level
        let base_price: i128 = (quantity as i128) * 100_000_000;
        let discounted = base_price * (100 - discount as i128) / 100;

        Ok(42u32) // actual order processing
    }
}
```



# Stellar & Soroban



Jão is using Soroban:

## 14) Jão upgrades the stand contract without losing any data

- The city introduces a new food safety regulation requiring Jão to log every baguncinha's cooking temperature.
- He needs to upgrade the contract to add this feature. Soroban contracts can be upgraded using `update_current_contract_wasm()`.
- The contract address stays the same.
- All existing storage is preserved.
- Clients don't need to update their addresses.
- But — there's no automatic rollback, so upgrades require careful migration logic.

Rust code :

```
#[contractimpl]
impl Baguncinha {
    /// Upgrade contract WASM - admin only
    pub fn upgrade( env: Env, caller: Address, new_wasm_hash: BytesN<32>, ) {
        caller.require_auth();
        let admin: Address = env.storage().instance().get(&DataKey::Admin).unwrap();
        assert_eq!(caller, admin, "Only admin can upgrade");
        // Replace WASM - takes effect immediately
        // All storage, address, and state is preserved
        env.deployer().update_current_contract_wasm(new_wasm_hash);
        // Emit upgrade event for indexers/dashboards
        env.events().publish( (symbol_short!("upgrade"),), new_wasm_hash, ); }

    /// Migration: run once after upgrade to transform old storage
    pub fn migrate_v2(env: Env, caller: Address) {
        caller.require_auth();
        let admin: Address = env.storage().instance()
            .get(&DataKey::Admin).unwrap();
        assert_eq!(caller, admin);
        // Check not already migrated
        assert!(
            !env.storage().instance().has(&symbol_short!("v2migr")),
            "Already migrated");
        // Add new version-2 config defaults
        env.storage().instance()
            .set(&symbol_short!("templog"), &true); // temperature logging
        env.storage().instance()
            .set(&symbol_short!("v2migr"), &true); // mark done
        env.storage().instance().extend_ttl(1000, 17280);
    }
}
```

# Stellar & Soroban



Jão is using Soroban:

## 15) Understanding and optimizing resource costs in Jão's contract

- Each Stellar transaction with a Soroban call has a resource budget: CPU instructions, memory bytes, read/write byte counts, and ledger entry accesses.
- Exceeding any limit causes a hard failure.
- The simulateTransaction RPC always runs first to calculate the exact fee.
- Jão learns to optimize: batch reads, minimize storage writes, cache values in local variables, and avoid loops over unbounded data.
- The fee model rewards efficient contracts.

Rust code :

```
/// ✅ OPTIMIZED: cache reads, minimize writes
pub fn optimized_sell(env: Env, customer: Address, qty: u32) {
    customer.require_auth();
    // Read ONCE into local var (avoid multiple storage reads)
    let stock: u32 =
env.storage().persistent().get(&DataKey::BaguncinhasLeft).unwrap_or(0);
    let total_sold: u32 =
env.storage().persistent().get(&DataKey::TotalSold).unwrap_or(0);
    assert!(qty <= stock, "Insufficient stock");
    // Compute ALL new values before writing
    let new_stock = stock - qty;
    let order_id = total_sold + 1;
    // Write ONCE per key (not in a loop)
    env.storage().persistent().set(&DataKey::BaguncinhasLeft, &new_stock);
    env.storage().persistent().set(&DataKey::TotalSold, &order_id);
    // Extend TTL for both keys in one sweep
    for key in [&DataKey::BaguncinhasLeft, &DataKey::TotalSold] {
        env.storage().persistent().extend_ttl(key, 17280, 120960);    }
}

/// ❌ WASTEFUL: multiple reads of same key
pub fn wasteful_sell(env: Env, customer: Address, qty: u32) {
    // Each .get() is a separate storage read - charges read fee
    assert!(env.storage().persistent().get(...) >= qty); // read 1
    env.storage().persistent().set(...,                // write 1
        env.storage().persistent().get(...) - qty);      // read 2!
}
```



# Stellar Bootcamp

Next class (26/5):

**Integration and Final dApp**  
(the last but not the least)